

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.18

RAW ACCESS

 Copy as Markdown

Raw access

Serverpod provides methods to execute raw SQL queries directly on the database for advanced scenarios. They are available on `session.db`.

`unsafeQuery`

Executes a single SQL query and returns a `DatabaseResult` containing the results. This method uses the extended query protocol, allowing for parameter binding to prevent SQL injection.

```
DatabaseResult result = await session.db.unsafeQuery(  
  r'SELECT * FROM mytable WHERE id = @id',  
  parameters: QueryParameters.named({'id': 1}),  
);
```

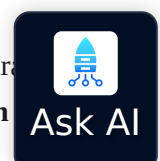
`unsafeExecute`

Executes a single SQL query without returning any results. Use this for statements that modify data, such as `INSERT`, `UPDATE`, or `DELETE`. Returns the number of rows affected.

```
int result = await session.db.unsafeExecute(  
  r'DELETE FROM mytable WHERE id = @id',  
  parameters: QueryParameters.named({'id': 1}),  
);
```

`unsafeSimpleQuery`

Similar to `unsafeQuery`, but uses the simple query protocol. This protocol does not support parameter binding, making it more susceptible to SQL injection. **Use with extreme caution and only when necessary.**



Simple query mode is suitable for:

- Queries containing multiple statements.
- Situations where the extended query protocol is not available (e.g., replication mode or with proxies like PGBouncer).

```
DatabaseResult result = await session.db.unsafeSimpleQuery(  
    r'SELECT * FROM mytable WHERE id = 1; SELECT * FROM othertable;',  
);
```

unsafeSimpleExecute

Similar to `unsafeExecute`, but uses the simple query protocol. It does not return any results. **Use with extreme caution and only when absolutely necessary.**

Simple query mode is suitable for the same scenarios as `unsafeSimpleQuery`.

```
int result = await session.db.unsafeSimpleExecute(  
    r'DELETE FROM mytable WHERE id = 1; DELETE FROM othertable;',  
);
```

INFO

Raw SQL is passed to the database as-is, so write your statements in the dialect of the **backend** your server runs against, PostgreSQL or SQLite. SQLite has no separate simple query protocol, and there the simple methods behave like their regular counterparts without parameter binding.

Query parameters

To protect against SQL injection attacks, always use query parameters when passing values into raw SQL queries. The library provides two types of query parameters:

- **Named parameters:** Use `@` to denote named parameters in your query and pass a `Map` of parameter names and values.
- **Positional parameters:** Use `$1`, `$2`, etc., to denote positional parameters and pass a `List` of parameter values in the correct order.

```
// Named parameters  
var result = await session.db.unsafeQuery(  
    r'SELECT id FROM apparel WHERE color = @color AND size = @size',  
    parameters: QueryParameters.named({  
        'color': 'green',  
    })
```

```

        'size': 'XL',
    }},
);

// Positional parameters
var result = await session.db.unsafeQuery(
    r'SELECT id FROM apparel WHERE color = $1 AND size = $2',
    parameters: QueryParameters.positional(['green', 'XL']),
);

```

⚠ DANGER

Always sanitize your input when using raw query methods. For the `unsafeQuery` and `unsafeExecute` methods, use query parameters to prevent SQL injection. Avoid using `unsafeSimpleQuery` and `unsafeSimpleExecute` unless the simple query protocol is strictly required.

Working with results

The query methods return a `DatabaseResult`, a read-only list of rows. Each row is itself a read-only list of column values, in the order the query selected them. Call `toColumnMap()` on a row to get the values keyed by column name instead.

```

var result = await session.db.unsafeQuery(
    r'SELECT id, name FROM company WHERE id = @id',
    parameters: QueryParameters.named({'id': 1}),
);

for (var row in result) {
    var id = row[0] as int;
    var map = row.toColumnMap(); // {'id': 1, 'name': 'Serverpod'}
}

```

Beyond the rows, the result exposes two properties:

- `affectedRowCount`: the number of rows affected by the query.
- `schema`: the columns of the result, each with a `columnName`.

When a query selects multiple columns with the same name, `toColumnMap()` keeps the last value for that name.

Intercept database access

To apply tracing, metrics, tenant scoping, or policy enforcement across every database operation of a session, including the raw methods above, see [Database interceptors](#).

